# Documentação de Engenharia de Software

Arquitetura de Análise de Redes Parlamentares Baseada em Teoria dos Grafos
para Identificação de Estruturas de Influência

Felipe Echeverria Vilhalva
Orientador: Prof. Dr. Rubens Barbosa Filho

2026

## 1 Visão Geral e Objetivos

Este capítulo apresenta uma proposta de documentação orientada à Engenharia de Software para o projeto de análise de redes parlamentares. O objetivo é garantir rastreabilidade entre problema, requisitos, implementação, testes e evidências experimentais.

### 1.1 Objetivo Geral

Projetar, implementar e validar uma arquitetura de software reprodutível para análise de redes parlamentares, fundamentada na teoria dos grafos, capaz de transformar dados legislativos brutos em evidências analíticas sobre estruturas de influência política. Para isso, o trabalho contempla a construção de um grafo ponderado de coautoria, o cálculo de métricas de centralidade, a detecção de comunidades e a análise temporal dos anos de 2015 a 2025, assegurando rastreabilidade entre requisitos, código, testes e artefatos gerados.

### 1.2 Objetivos Específicos

1. **Modelagem do grafo de coautoria:** Implementar um grafo ponderado de coautoria entre deputados, com regras de consistência para agregação de arestas, remoção de inconsistências e suporte a filtros por ano, partido e unidade federativa.

2. **Cálculo de métricas de centralidade:** Implementar as métricas de degree, betweenness, closeness e eigenvector centrality, produzindo saídas estruturadas por deputado e por ano.

3. **Detecção de comunidades:** Implementar ao menos dois algoritmos de detecção de comunidades (ex.: Louvain e Label Propagation), com comparação por modularidade e interpretação política dos agrupamentos.

4. **Análise temporal:** Executar o pipeline anual (2015-2025) para observar evolução de propriedades globais da rede.

5. **Qualidade e reprodutibilidade:** Garantir contratos claros de entrada e saída no core, cobertura mínima de testes automatizados e execução reprodutível via comando único.

# 2 Escopo e Arquitetura

## 2.1 Delimitação do Escopo e Estrutura de Diretórios

O projeto é estruturado em camadas modulares baseadas nos princípios da Arquitetura Limpa [4], isolando as regras de negócio das tecnologias externas. O código-fonte principal, concentrado no diretório `src/`, é fisicamente separado dos artefatos gerados no diretório `data/`, garantindo a reprodutibilidade e a integridade do pipeline.

A arquitetura de software implementada é dividida nos seguintes módulos principais:

- `models`: Define as entidades de domínio independentes de frameworks, agrupando as classes fundamentais em arquivos como `deputado.py`, `proposicao.py` e `aresta_coautoria.py`, além da estrutura central em `rede_parlamentar.py`.
- `core`: Concentra os casos de uso da aplicação. O arquivo `graph.py` orquestra a estrutura de dados, enquanto o subdiretório `algorithms/` abriga a matemática pesada em `community_detection.py` e `metrics.py`.
- `extraction` e `processing`: Atuam como adaptadores de entrada. O módulo `camara_extrator.py` consome a API oficial, enquanto o `limpeza.py` padroniza e trata inconsistências nos dados brutos.
- `repository`: Camada de persistência que abstrai as operações de I/O através de implementações como `csv_repository.py`, `db_repository.py` e `graph_exporter.py`. Ela alimenta o banco de dados `parliament.db` e gera as exportações finais, como o `grafo_camara_2025.gexf` e o `deputados_metricas_2025.csv`.
- `visualization`: Responsável pela camada de apresentação, utilizando o `plots.py` para consumir os resultados e gerar análises gráficas (ex: `correlacao_centralidades.png`) no diretório `data/plots/`.
- `tests`: Garante a estabilidade de cada módulo de forma isolada, com arquivos de teste unitário espelhando a arquitetura, como `test_graph.py`, `test_processing.py` e `test_repository.py`.

## 2.2 Stack Tecnológica e Infraestrutura

O desenvolvimento e a execução do projeto baseiam-se em um ecossistema focado em reprodutibilidade, utilizando **Python** como linguagem principal de programação. Para garantir o isolamento do ambiente e o cumprimento da meta de execução via comando único, a infraestrutura é totalmente conteinerizada utilizando **Docker** e orquestrada via **Docker Compose**.

As principais bibliotecas adotadas para sustentar a arquitetura incluem:

- **Extração e Processamento:** `requests` para o consumo da API da Câmara, em conjunto com `pandas` e `pyarrow` para a manipulação eficiente dos dados tabulares.

- **Grafos e Matemática:** `networkx` para a modelagem das estruturas de rede e cálculos topológicos, com suporte numérico do `scipy`.
- **Visualização:** `matplotlib` e `seaborn` para a plotagem dos artefatos gráficos e distribuições de centralidade.
- **Persistência e Configuração:** `pysqlite3-binary` para o banco de dados local da camada repository, e `python-dotenv` para o gerenciamento seguro de variáveis de ambiente.
- **Versionamento e Colaboração:** `Git` e `GitHub` para controle de versão, rastreabilidade das mudanças e colaboração no desenvolvimento.
- **Qualidade e Testes:** `pytest` e `pytest-cov` para garantir a validação contínua e a cobertura do core da aplicação.

### 2.3 Notas sobre a Implementação Contínua

Como o projeto encontra-se em fase ativa de desenvolvimento, a implementação técnica detalhada na subseção anterior está sujeita a ciclos de refatoração. Destaca-se que:

- A nomenclatura atual de arquivos, diretórios e variáveis de escopo interno poderá ser padronizada (visando eliminar a mistura de termos em inglês e português), adotando-se convenções de código limpo.
- Detalhes específicos da divisão de métodos e classes poderão sofrer ajustes pontuais para melhorar a coesão e o acoplamento do sistema, sem alterar as responsabilidades de cada camada arquitetural.
- A natureza agnóstica dos artefatos exportados pela camada de repositório (arquivos CSV e GEXF) garante que a fase de análise de dados e geração de gráficos não fique acoplada exclusivamente ao ecossistema Python. Caso necessário, a arquitetura permite a fácil integração de ferramentas estatísticas especializadas adicionais, como a linguagem R, para a etapa de análises macro e longitudinais.

## 3 Metas de Execução

- **Meta mínima (obrigatória):** Grafo ponderado + métricas de centralidade + testes unitários do core.
- **Meta esperada:** Inclusão de dois algoritmos de comunidade + comparação quantitativa.
- **Meta de diferenciação (se houver tempo):** Análise temporal completa 2015-2025 com discussão longitudinal dos resultados.

## 4 Cronograma de Atividades

As atividades estão distribuídas de abril a novembro de 2026, com entregas críticas em outubro e defesa em novembro.

| | Meses de 2026 | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Plano de atividades** | **04** | **05** | **06** | **07** | **08** | **09** | **10** | **11** |
| **1. Revisão Bibliográfica** | X | | | | | | | |
| **2. Desenvolvimento Arq.** | X | X | | | | | | |
| **3. Execução e Testes** | | X | X | | | | | |
| **4. Análise Topológica** | | | X | X | X | | | |
| **5. Análise Longitudinal** | | | | X | X | | | |
| **6. Revisão, Entregas e Defesa** | | | | | | X | X | X |
| **7. Escrita da Monografia** | X | X | X | X | X | X | X | |

Tabela 1: Cronograma de execução das atividades do Projeto Final de Curso.

# 5 Validação e Rastreabilidade

## 5.1 Critérios de Aceitação

1. O core deve construir corretamente o grafo de coautoria, sem arestas duplicadas não agregadas.
2. As métricas de centralidade devem ser calculadas e exportadas em formato tabular por deputado.
3. Ao menos dois métodos de comunidades devem gerar partições válidas e comparáveis por modularidade.
4. O pipeline anual deve produzir série histórica consolidada para os principais indicadores globais.
5. O processo completo deve ser reproduzível com documentação e execução padronizada, gerando ao menos 4 gráficos e 1 resumo textual.

## 5.2 Matriz de Rastreabilidade Funcional

| ID | Requisito Funcional | Validação | Evidência |
|---|---|---|---|
| RF-01 | Coletar dados legislativos por ano | Testes de extração e consistência dos campos | Base bruta anual consolidada |
| RF-02 | Construir grafo de coautoria ponderado | Testes de estrutura e de agregação de arestas | Grafo final em formato GEXF |
| RF-03 | Calcular métricas de centralidade por deputado | Testes unitários dos cálculos e checagem de faixas | Tabela de métricas em CSV |
| RF-04 | Detectar comunidades na rede | Execução de algoritmos e comparação por modularidade | Partições e valor de modularidade |
| RF-05 | Gerar saídas analíticas do pipeline | Execução fim a fim do fluxo anual | Gráficos e resumo textual |

Tabela 2: Matriz de rastreabilidade requisito-implementação-validação.

## 5.3 Matriz de Rastreabilidade Não Funcional

| ID | Requisito Não Funcional | Validação | Evidência |
|---|---|---|---|
| RNF-01 | Reprodutibilidade do ambiente e da execução | Execução completa em ambiente limpo com comando único | Geração dos artefatos esperados |
| RNF-02 | Eficiência de processamento do pipeline anual | Medição de tempo e memória por ano processado | Resumo de desempenho consolidado |
| RNF-03 | Testabilidade e confiabilidade do core | Execução da suíte automatizada com cobertura mínima | Relatório de testes e cobertura |
| RNF-04 | Manutenibilidade da arquitetura em camadas | Revisão de coesão e acoplamento entre módulos | Estrutura modular documentada |
| RNF-05 | Interoperabilidade dos resultados exportados | Leitura dos artefatos em ferramentas externas | Arquivos CSV e GEXF válidos |

Tabela 3: Matriz de rastreabilidade de requisitos não-funcionais.

# 6 Formulações Matemáticas

## 6.1 Grau Ponderado

Seja $G = (V, E)$ um grafo não direcionado e ponderado [3], com peso $w_{ij}$ na aresta entre os vértices $i$ e $j$. O grau ponderado de um vértice $i$ é:

$$s_i = \sum_{j \in N(i)} w_{ij} \tag{1}$$

onde $N(i)$ representa os vizinhos de $i$.

## 6.2 Centralidade de Grau Normalizada

No projeto, a normalização do grau ponderado é feita pela força total da rede [5]:

$$C_D(i) = \frac{s_i}{\sum_{k \in V} s_k} \tag{2}$$

## 6.3 Betweenness Centrality

A centralidade de intermediação de um vértice $v$ [5, 2] é definida por:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \tag{3}$$

onde $\sigma_{st}$ é o número de caminhos mínimos entre $s$ e $t$, e $\sigma_{st}(v)$ é o número desses caminhos que passam por $v$.

## 6.4 Densidade do Grafo

Para um grafo não direcionado com $n = |V|$ vértices e $m = |E|$ arestas [1]:

$$\rho(G) = \frac{2m}{n(n-1)} \tag{4}$$

## 6.5 Modularidade (Q)

A modularidade é uma métrica que avalia a qualidade da divisão de uma rede em comunidades [5]. Ela compara a densidade de arestas dentro das comunidades com a densidade esperada em um grafo aleatório equivalente. É definida matematicamente por:

$$Q = \frac{1}{2m} \sum_{i,j} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{5}$$

Onde:

- $A_{ij}$ é o peso da aresta entre os nós $i$ e $j$.
- $k_i$ e $k_j$ são os graus (ou graus ponderados) dos nós $i$ e $j$.
- $m$ é a soma dos pesos de todas as arestas da rede.
- $c_i$ e $c_j$ são as comunidades às quais os nós $i$ e $j$ pertencem.
- $\delta(c_i, c_j)$ é a função delta de Kronecker, que resulta em 1 se $c_i = c_j$ e 0 caso contrário.

## 6.6 Algoritmo de Louvain

O algoritmo de Louvain é um método heurístico eficiente para maximizar a modularidade $Q$ [1]. Ele opera em duas fases que se repetem iterativamente:

1. **Otimização Local:** Inicialmente, cada nó é atribuído a uma comunidade distinta. Para cada nó $i$, avalia-se o ganho de modularidade ($\Delta Q$) ao movê-lo para a comunidade de um de seus vizinhos $j$. O nó é movido para a comunidade que oferece o maior aumento positivo em $Q$. Este processo se repete sequencialmente até que nenhum movimento individual consiga aumentar a modularidade.

2. **Agregação:** Uma nova rede macroscópica é construída. As comunidades encontradas na fase anterior tornam-se os nós desta nova rede (super-nós). Os pesos das arestas entre os super-nós são definidos pela soma dos pesos das arestas entre os nós das respectivas comunidades originais.

As duas fases são repetidas até que haja estabilização e não seja possível obter ganhos adicionais na modularidade da rede.

# 7 Ameaças à Validade e Mitigação

- **Viés de representação e dependência de regras de peso:** Para evitar a distorção causada por assinaturas em massa e refletir a articulação política real, adotou-se uma heurística de pesos proporcionais à relevância institucional e ao quórum exigido para cada tipo de proposição: Projetos de Lei (PL) possuem peso 10, Projetos de Lei Complementar (PLP) possuem peso 5, e Propostas de Emenda à Constituição (PEC) possuem peso 1. Além disso, se necessário, vai ser possivel alterar os pesos manualmente .

# Referências


[1] BARABÁSI, A.-L. *Network Science*. Cambridge: Cambridge University Press, 2016.

[2] CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.

[3] DIESTEL, R. *Graph Theory*. 5. ed. Berlin: Springer, 2017.

[4] MARTIN, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. 1. ed. Boston: Prentice Hall, 2017.

[5] NEWMAN, M. E. J. *Networks: An Introduction*. Oxford: Oxford University Press, 2010.